

wokwi 專案開發歷

AI 實務協作與心得

人機協作體系：專業建議與 AI 執行



AI 操作員 (Nick)

負責主導開發方向、審核 AI 產出，並下達精確指令引導開發進程。



專業顧問團隊

Leong, Wilbur 提供深厚的嵌入式專業建議，引導關鍵技術決策（如：建議採用硬體 I2C 控制器）。



執行核心

Gemini CLI 擔任「嵌入式開發專家」角色，負責底層實作、高速除錯與文件化。

超越對話框：從「複製貼上」到「自動化代理」

📁 傳統 Web AI 痛點

📄 手動搬運：

開發者需不斷在 IDE 與瀏覽器間複製貼上程式碼，繁瑣且易錯。

🔗 脈絡斷層：

無法感知整個專案的檔案結構與文件，容易給出不符現狀的建議。

📄 排版破壞：

AI 經常重寫整份檔案，導致原有的精細排版遺失，開發者需手動比對差異 (Diff)。

> Gemini CLI (Auto-Edit) 優勢

📄 主動環境感知：

直接讀取 readme.md 與工作區源碼，建立零誤差的全域上下文。

✂️ 手術式修改 (Surgical Edit)：

精準鎖定特定行數進行區塊替換，完美保留其餘排版與註解。

🔗 系統級整合：

修改後能直接觸發 Shell 指令，甚至自動生成結構化的 Git Commit 紀錄。

奠定工程紀律：指定版本規範 (Commit: b8e26a)

角色定位

專業嵌入式開發專家，專注高效能、低延遲的底層解決方案。

溝通規則

繁體中文說明，英文術語/符號，風格精簡直接，不囉嗦！

開發流程

需求分析 → 設計規劃 → 實作驗證 → 紀錄 (含 User Prompt)。

特定規則

硬體安全優先、符合業界標準、Git 紀錄與開發歷程隨時同步。

關鍵除錯實錄一：解決 RTC 頑疾

困境 & 決策點

⚠ 系統困境：

RTC (DS1307) 在主程式中持續讀取失敗 (ERR)，通訊狀態陷入不明。

🔧 關鍵決策：

絕不盲目修改主邏輯！要求 AI 撰寫獨立測試程式 (hw_i2c_debug.ino) 進行隔離診斷。

除錯成果

- 🔍 透過測試程式進行暫存器掃描 (Register Dump)，排除雜訊干擾。
- ✂ 發現「多位元組讀取時 ACK/STOP 順序不合規」與「冷啟動無效日期」核心問題。
- ✂ 在隔離環境中測通後，再**手術式回填**至主系統。

關鍵除錯實錄二：時序精準度

困境與 AI 盲點

系統困境：

畫面更新頻率無法精準維持在每秒一次，出現時間漂移與不穩定。

AI 的盲點：

預設使用軟體定時器 (millis()) 進行排程，容易受到主迴圈其他任務阻塞而產生延遲。

領域知識的價值

專家決策：

使用者運用領域知識，主動提示 AI 應改用 **RTC (硬體時鐘)** 的資料作為時間基準來觸發更新。

核心洞察 (Insight)：

若缺乏足夠的硬體領域知識來給予正確的技術提示，便難以引導 AI 突破盲點，發揮其最大價值。

從規則到成品：結構化的 AI 協作

2

4

基本規則

透過 GEMINI.md 確立開發風格與底層規範。

架構規劃

寫程式前先要求架構分析 (Strategy)。

反覆 Debug

根據現象與 Log, 引導 AI 深入底層修正。

提供需求

給予詳細 readme.md 與硬體配置表。

開始實作

嚴格遵守 Bottom-Up 進行分層開發。

1

3

5

開發心得總結：快、狠、準



快

AI 實作底層暫存器與大量格式化字串的速度，遠遠超越傳統手寫代碼。



狠

精確切入硬體手冊 (RM0008) 規範，果斷捨棄臃腫緩慢的高階函式庫。



準

「有明確的方向，可以更快解決問題。」精準的 Prompt 是成功的最大關鍵。

總結：AI 是一個強大的執行器，搭配人類的專業判斷與精確的方向引導，能大幅縮短嵌入式系統的開發週期！

放眼未來：結合工作與團隊賦能



累積與共享

詳細記錄每一次的 Prompt 決策，將除錯歷程沉澱為文件，便於團隊內部知識共享與技術傳承。



持續迭代 Prompt

根據實務遇到的盲點，持續將新建立的開發規範更新至 GEMINI.md，打造越用越懂專案的 AI 助理。



融入日常開發

讓 AI 從單純的「問答工具箱」進化為具備專案脈絡記憶的「虛擬共同開發者」，真正實現工作產能的飛躍。